

Module 1: Spring Core and Maven

Exercise 1: Configuring a Basic Spring Application

Topics

- What is Spring Framework?
- Spring Container
- Bean
- XML Configuration
- ApplicationContext

Example:

```
<!-- applicationContext.xml -->
```

```
<bean id="student" class="com.example.Student"/>
```

```
ApplicationContext context =
```

```
    new ClassPathXmlApplicationContext("applicationContext.xml");
```

```
Student s = context.getBean("student", Student.class);
```

Learn

- Spring IoC Container
 - Bean creation
 - Loading XML configuration
-

Exercise 2: Implementing Dependency Injection

Instead of creating objects manually:

```
Student s = new Student();
```

```
Address a = new Address();
```

```
s.setAddress(a);
```

Spring creates and injects dependencies.

Example:

```
<bean id="address" class="com.example.Address"/>
```

```
<bean id="student" class="com.example.Student">
```

```
  <property name="address" ref="address"/>
```

```
</bean>
```

Concepts

- Dependency Injection (DI)
 - Loose Coupling
 - IoC
-

Exercise 4: Creating and Configuring a Maven Project

Maven Directory Structure

Project

```
|
|
|└─ src
|
|  └─ main
|
|    └─ java
|
|      └─ resources
|
|    └─ test
|
|
└─ pom.xml
```

Sample pom.xml

<dependencies>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>6.1.5</version>

</dependency>

</dependencies>

Know:

- dependency
- groupId
- artifactId
- version

Exercise 5: Configuring the Spring IoC Container

IoC = Inversion of Control

Instead of

```
Student s = new Student();
```

Spring creates the object.

Configuration can be done using

- XML
- Java Configuration
- Annotation

Example

@Configuration

```
public class AppConfig {
```

```
    @Bean
```

```
    public Student student() {
```

```
        return new Student();
```

```
    }
```

```
}
```

Exercise 7: Constructor and Setter Injection

Constructor Injection

```
public class Student {  
  
    private Address address;  
  
    public Student(Address address) {  
        this.address = address;  
    }  
  
}
```

XML

```
<constructor-arg ref="address"/>
```

Setter Injection

```
public class Student {  
  
    private Address address;  
  
    public void setAddress(Address address) {  
        this.address = address;  
    }  
  
}
```

XML

```
<property name="address" ref="address"/>
```

Difference:

Constructor Injection

Setter Injection

Mandatory dependency

Optional dependency

Object created fully initialized

Can be modified later

More secure

More flexible

Exercise 9: Creating a Spring Boot Application

Typical Structure

src

```
└─ main
    └─ java
        |   Application.java
    └─ resources
        application.properties
```

Main class

@SpringBootApplication

```
public class DemoApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(DemoApplication.class, args);
```

```
    }
```

```
}
```